Text-to-SQL con QLoRA en producción

Generación segura de consultas MySQL desde preguntas en español

Objetivo

Poner en producción el modelo desarrollado en el Proyecto 3 para que una persona externa pueda usarlo mediante una aplicación web. La adaptación conserva los datos, el modelo y las métricas, y añade validación de entradas, guardia SQL, telemetría privada, despliegue reproducible y evidencia operativa verificable.

Wilson Alejandro Calderón Argueta 22018
Pablo Daniel Barillas Moreno 22193

Deep Learning y Sistemas Inteligentes
2026

Repositorio:
https://github.com/DanielBarillasM/Proyecto-Final_Grupo-1_DLYSI_Sec-30

Índice

1. Resumen ejecutivo . . . 3
2. Problemática y alcance de la decisión . . . 3
   - 2.1. Situación actual . . . 3
   - 2.2. Usuario y flujo esperado . . . 4
   - 2.3. Casos incluidos y exclusiones . . . 4
3. Planificación y gobierno del trabajo . . . 4
   - 3.1. Etapas y puertas de calidad . . . 4
   - 3.2. Responsabilidades y control de cambios . . . 5
   - 3.3. Riesgos del plan . . . 5
4. Supuestos y criterios de uso . . . 5
   - 4.1. Supuestos de entrada . . . 5
   - 4.2. Supuestos de operación . . . 5
5. Procedencia y gobierno de datos . . . 6
   - 5.1. Proyecto de origen . . . 6
   - 5.2. Dataset congelado . . . 6
   - 5.3. Regla de trazabilidad . . . 6
6. Modelo planteado . . . 7
   - 6.1. Modelo base y adaptación QLoRA . . . 7
   - 6.2. Flujo de inferencia . . . 7
   - 6.3. Cambios introducidos para producción . . . 7
7. Validación de hipótesis . . . 8
   - 7.1. Hipótesis y criterios previos . . . 8
   - 7.2. Interpretación del resultado . . . 8
8. Herramientas y arquitectura de producción . . . 9
   - 8.1. Vista por capas . . . 9
   - 8.2. Responsabilidad de cada componente . . . 9
   - 8.3. Secuencia de una solicitud . . . 9
9. Seguridad, privacidad y observabilidad . . . 10
   - 9.1. Modelo de amenazas . . . 10
   - 9.2. Guardia de salida . . . 10

9.3. Telemetría privada . . . 10
10. Decisiones de despliegue . . . 11
10.1. Alternativas evaluadas . . . 11
10.2. Ruta permanente preparada . . . 11
11. Evidencias del pase a producción . . . 11
11.1. Prueba pública del 5 de noviembre . . . 11
11.2. Capturas y cierre controlado . . . 12
11.3. Inventario de evidencia . . . 13
12. Verificación y reproducibilidad . . . 13
12.1. Batería de calidad . . . 13
12.2. Preparación desde una clonación limpia . . . 13
12.3. Comandos de auditoría . . . 14
13. Limitaciones, operación y trabajo futuro . . . 14
13.1. Limitaciones conocidas . . . 14
13.2. Condiciones para mantenerlo en producción . . . 14
13.3. Próximos pasos priorizados . . . 14
14. Conclusiones . . . 15
Uso de inteligencia artificial . . . 15
Mapa de artefactos . . . 15

1. Resumen ejecutivo

Este proyecto convierte un prototipo académico de Text-to-SQL en un servicio demostrable por una persona ajena al equipo. El sistema recibe el DDL de un esquema relacional y una pregunta en español, y devuelve una única consulta MySQL de solo lectura. La propuesta no intenta reemplazar al analista ni ejecutar cambios sobre una base de datos: reduce la barrera de escribir SQL y mantiene una revisión humana antes de cualquier ejecución.

El modelo base es `Qwen2.5-Coder-1.5B-Instruct` y la adaptación seleccionada es QLoRA `r=32`. En la validación heredada del Proyecto 3 obtuvo 97.4 % de exactitud ponderada por ejecución y 97.5 % de consultas ejecutables, frente a 3.8 % y 62.5 % del modelo base. Las dos configuraciones QLoRA empataron en calidad; `r=32` se eligió por su menor latencia p95.

El aporte del Proyecto Final está en la ingeniería de producción. Se implementaron una interfaz Next.js, una API FastAPI, recuperación reproducible del adaptador, límites de entrada, validación de SQL, control de concurrencia, telemetría sin contenido de usuario y scripts para abrir, verificar y cerrar una demostración pública. La ruta pública temporal usa Cloudflare Quick Tunnel y conserva el modelo en una RTX 4050 local.

El 5 de noviembre de 2026 la aplicación respondió HTTP 200 desde una sesión pública, generó tres consultas sintéticas correctas y rechazó dos casos destructivos con HTTP 422. Después de capturar la evidencia, el túnel y los procesos fueron cerrados y se verificó que los puertos quedaran libres. La entrega se complementa con 43 pruebas automatizadas, análisis estático, compilación de la aplicación web, hashes de datos y modelo, capturas, registros anonimizados y un paquete mínimo para un futuro Hugging Face Space.

Resultado principal

El proyecto demuestra que el modelo puede mantenerse para consultas supervisadas siempre que se conserven cuatro condiciones: salida de solo lectura, revisión humana, telemetría privada y monitoreo de errores, latencia y cambios de dominio.

2. Problemática y alcance de la decisión

2.1. Situación actual

Una persona que conoce el negocio y la estructura de sus datos no necesariamente domina SQL. Para responder una pregunta operativa suele describir la necesidad a un especialista, esperar que este interprete el esquema, escriba la consulta y devuelva el resultado. Esta dependencia incrementa el tiempo de respuesta incluso cuando la pregunta solo requiere una consulta de lectura.

El sistema reduce esa fricción: el usuario proporciona el esquema en DDL y formula la pregunta en español. El modelo propone SQL que puede revisarse, copiarse y validarse. La decisión apoyada no es "qué acción debe ejecutar la organización", sino "qué consulta de lectura representa mejor la pregunta planteada sobre este esquema".

2.2. Usuario y flujo esperado

El usuario objetivo es un analista, encargado operativo o desarrollador con autorización para consultar datos, conocimiento de las entidades del negocio y capacidad de revisar la consulta antes de ejecutarla. El flujo previsto es:

1. pegar un esquema MySQL que contenga las tablas y relaciones relevantes;
2. escribir una pregunta concreta en español;
3. solicitar una generación y esperar la respuesta del modelo;
4. revisar tablas, filtros, agregaciones, uniones y límites;
5. ejecutar la consulta únicamente en un entorno con permisos apropiados.

Alcance operativo

La aplicación propone SQL para revisión. No solicita credenciales, no se conecta a una base de negocio y no ejecuta cambios. Una consulta sintácticamente segura todavía puede ser semánticamente incorrecta, por lo que el usuario debe revisarla antes de ejecutarla.

2.3. Casos incluidos y exclusiones

El alcance incluye selección, filtros, ordenamiento, agregaciones, uniones y expresiones comunes de MySQL. La demostración cubre una pregunta fácil, una media y una difícil sobre un esquema sintético de clientes y pedidos. Quedan fuera la ejecución directa, las credenciales, los datos personales, las operaciones DDL o DML, las múltiples sentencias y la automatización de decisiones de negocio.

3. Planificación y gobierno del trabajo

3.1. Etapas y puertas de calidad

Fecha	Resultado	Puerta de calidad
2–3 nov	Migración y trazabilidad	Código, datos y hashes verificados
4 nov	Aplicación y controles	Flujo local, guardia SQL y telemetría
5 nov	Despliegue	HTTP 200 público, tres casos y cierre verificado
5 nov	Evidencias y documentación	README, registros, capturas y pruebas
5 nov	Presentación e informe	PDF de diez diapositivas e informe compilado
9 nov	Congelamiento	Release candidata y contingencia local
10–15 nov	Ajustes	Correcciones sin alterar el experimento

La planificación separó el trabajo heredado del Proyecto 3 de las tareas nuevas de producción. Las métricas y los datos no se regeneraron; primero se verificó su procedencia y luego se construyeron el servicio, los controles y la evidencia. Esta separación evita presentar como nueva una mejora que no fue entrenada durante el Proyecto Final.

3.2. Responsabilidades y control de cambios

Pablo mantiene el runtime, la interfaz, el despliegue, las pruebas y la documentación. Wilson realiza una auditoría independiente y ambos integrantes preparan la exposición oral. Los cambios se publican en commits pequeños, mientras que los pesos, cachés, entornos virtuales, logs privados y archivos temporales permanecen fuera de Git.

El repositorio heredado identifica dos integrantes. El enunciado general menciona modalidad de tríos, pero no se añadió una tercera persona ficticia ni se le atribuyó trabajo. Esta diferencia debe confirmarse administrativamente con el docente y no altera el contenido técnico del proyecto.

3.3. Riesgos del plan

Riesgo	Mitigación
Descarga lenta del modelo	Revisiones inmutables, caché local y calentamiento antes de exponer
Falta de GPU pública	Demostración en RTX 4050 y paquete preparado para ZeroGPU
SQL destructivo	Análisis de sentencia única y lista de operaciones permitidas
Fuga de información	Logs sin pregunta, DDL, credenciales ni datos personales
Servicio inaccesible	Capturas, evidencia estructurada y modo local como contingencia
Cambio de dominio	Uso supervisado y seguimiento de errores por dominio

4. Supuestos y criterios de uso

4.1. Supuestos de entrada

- El usuario proporciona un DDL con al menos una sentencia `CREATE TABLE`.
- La pregunta está en español y se refiere a entidades presentes en el esquema.
- El dialecto objetivo es MySQL 8.4 y la salida debe ser una sola consulta de lectura.
- El esquema tiene información suficiente sobre claves, columnas y relaciones relevantes.
- El usuario puede reconocer si la consulta propuesta responde su intención.

4.2. Supuestos de operación

- La demostración usa una RTX 4050 local con un entorno CUDA compatible.
- La primera solicitud incluye la carga del modelo y puede ser notablemente más lenta.
- El servicio procesa una generación a la vez para proteger memoria y estado del runtime.
- La URL de Quick Tunnel es temporal y cambia cada vez que se inicia la demostración.

- Los datos sintéticos y dos esquemas inéditos no representan todos los dominios reales.

Interpretación correcta de la seguridad

"Solo lectura" significa que la guardia no permite una operación de escritura. No significa que la consulta sea necesariamente correcta, eficiente o autorizada para todos los usuarios. La revisión humana y los permisos de la base siguen siendo controles obligatorios.

5. Procedencia y gobierno de datos

5.1. Proyecto de origen

La base técnica proviene del Proyecto 3 en el commit `df34c14c5afecddaa0d977832a56cec80c590244`. La migración excluyó el historial Git original, entornos virtuales, cachés, builds y temporales, pero conservó los datos, resultados, configuración, scripts relevantes, machotes y evidencia experimental.

5.2. Dataset congelado

Elemento	Cantidad
Pares de entrenamiento	2,592
Esquemas de entrenamiento	12
Preguntas de validación	40
Esquemas inéditos de validación	2
Preguntas fáciles / medias / difíciles	14 / 14 / 12
Filas descartadas	0
Filas con revisión humana	100

La validación se ejecutó en MySQL 8.4.10 con modo estricto. Los conteos y hashes se conservan en `resultados/dataset_manifest.json`. El SHA-256 del conjunto de entrenamiento comienza con `9e28bcd7` y el de validación con `e1af4f01`. Congelar estos valores permite distinguir una corrección de ingeniería de una alteración del experimento.

5.3. Regla de trazabilidad

Cada afirmación tiene una fuente definida: los datos se rastrean a `datos/`, las métricas a `resultados/`, las revisiones del modelo a `config.json` y `adapter/registry.json`, y los hechos de producción a `docs/evidencias/`. Ninguna URL, captura o latencia se presenta como real si no fue producida por el servicio ejecutado.

6. Modelo planteado

6.1. Modelo base y adaptación QLoRA

El modelo base es `Qwen/Qwen2.5-Coder-1.5B-Instruct`, fijado a la revisión `2e1fd397...f1098a`. Se escogió por estar orientado a código, tener un tamaño manejable para una GPU portátil y permitir adaptación eficiente mediante PEFT.

QLoRA conserva el modelo cuantizado y entrena matrices de bajo rango en módulos seleccionados. Esto reduce el costo de ajuste respecto de entrenar todos los parámetros. Se compararon dos rangos: `r=16`, con 18,464,768 parámetros entrenables, y `r=32`, con 36,929,536. Sus tiempos de entrenamiento fueron aproximadamente 120.7 y 133.4 minutos, respectivamente.

El adaptador seleccionado se publica en el repositorio público de Hugging Face y se fija a la revisión `24658a4e...6de84`. El peso `adapter_model.safetensors` mide aproximadamente 140.9 MB y su SHA-256 comienza con `d3126183`. El repositorio no duplica este binario: el runtime lo descarga y verifica, o utiliza una copia local coincidente.

6.2. Flujo de inferencia

1. se normalizan y validan el esquema y la pregunta;
2. el prompt combina instrucciones, DDL y lenguaje natural;
3. el runtime carga el modelo base y aplica el adaptador fijado;
4. la generación produce un candidato de SQL de forma determinista;
5. la salida se limpia y analiza con `sqlglot`;
6. si es una única consulta de lectura, se devuelve junto con metadatos técnicos;
7. si viola el contrato, se genera un error visible y categorizado.

La ruta experimental también evaluó una reparación única ante error SQL. La exactitud ponderada fue 97.4 % tanto en modo directo como con reparación, por lo que la producción no depende de una mejora no observada.

6.3. Cambios introducidos para producción

- descarga reproducible del adaptador cuando no existen pesos locales;
- interfaz Next.js y alternativa Gradio empaquetada para Spaces;
- API con validación de campos, longitudes y presencia de `CREATE TABLE`;
- guardia que acepta únicamente `SELECT` o `WITH ... SELECT`;
- respuesta con latencia, variante e identificador de solicitud;
- telemetría que omite el esquema y la pregunta;
- scripts de apertura, verificación y cierre seguro de la demostración.

7. Validación de hipótesis

7.1. Hipótesis y criterios previos

Antes del entrenamiento se definieron dos criterios de éxito principales: exactitud ponderada por ejecución de al menos 0.70 y mejora absoluta de al menos 0.15 frente al baseline. Además se esperaba que `r=32` mejorara la exactitud de `r=16`; si existía empate, la decisión usaría ejecutabilidad y latencia p95.

La métrica EX compara el resultado de ejecutar la consulta generada con el resultado de referencia. La versión ponderada evita que el promedio oculte la distribución de dificultad. La tasa ejecutable mide si MySQL acepta la consulta, pero por sí sola no comprueba que responda correctamente.

Variante	EX ponderada	Ejecutable	p95	Entrenamiento
Qwen base	3.8 %	62.5 %	6.66 s	–
QLoRA $r = 16$	97.4 %	97.5 %	15.72 s	120.7 min
QLoRA $r = 32$	**97.4 %**	**97.5 %**	**13.20 s**	**133.4 min**

7.2. Interpretación del resultado

La hipótesis de superioridad en exactitud para `r=32` no se confirmó: ambas configuraciones obtuvieron 39 respuestas correctas de 40. Sin embargo, las dos superaron ampliamente el umbral de 0.70 y la mejora mínima frente al baseline. Después del empate se seleccionó `r=32` porque su latencia p95 fue aproximadamente 2.5 segundos menor.

El baseline produjo respuestas más rápidas, pero su 3.8 % de EX ponderada demuestra que velocidad sin calidad no satisface la decisión. La tasa ejecutable de 62.5 % también indica que una parte considerable de sus salidas ni siquiera podía ejecutarse. En contraste, QLoRA combina alta calidad y 97.5 % de ejecutabilidad con un costo de latencia aceptable para uso supervisado.

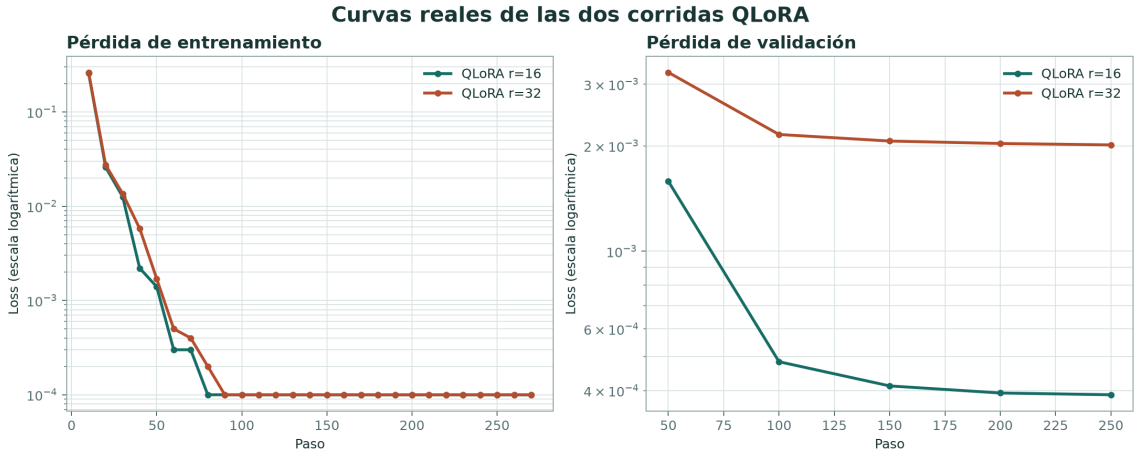
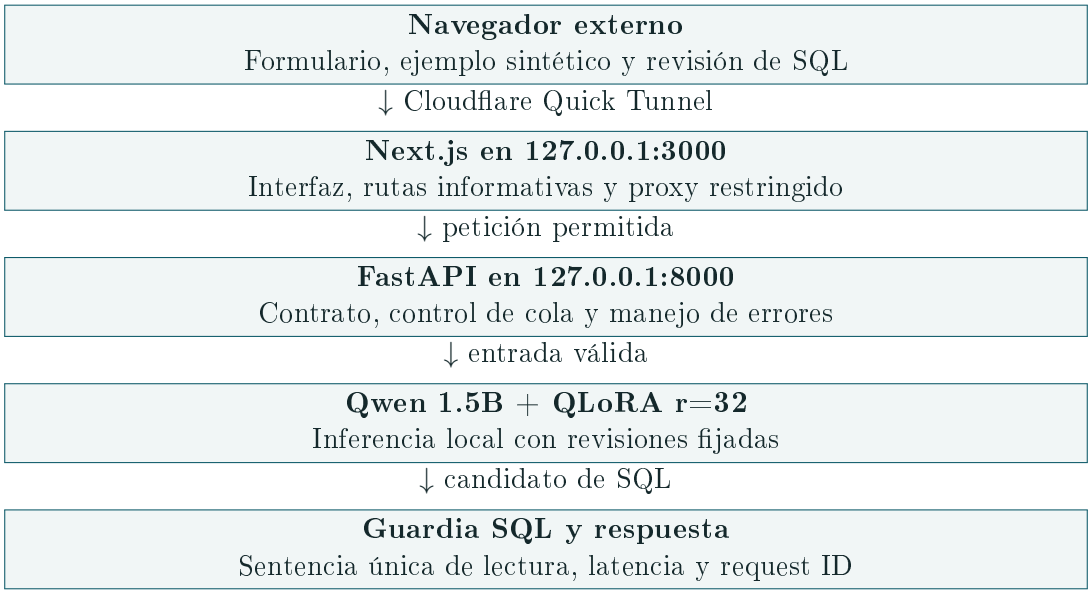


Figura 1: Pérdida de entrenamiento y validación de las dos configuraciones QLoRA. La selección final se basa en ejecución y latencia, no únicamente en la curva de pérdida.

Las curvas confirman convergencia, pero no justifican por sí solas la selección. El criterio operativo se apoya en consultas ejecutadas sobre esquemas inéditos, porque el objetivo final es producir SQL útil y no solo minimizar una pérdida de entrenamiento.

8. Herramientas y arquitectura de producción

8.1. Vista por capas



8.2. Responsabilidad de cada componente

Capa	Herramientas	Responsabilidad
Datos	JSONL, MySQL 8.4	Pares congelados, validación ejecutable y hashes
Modelo	PyTorch, Transformers, PEFT	Carga del modelo, adaptador e inferencia
API	FastAPI, Pydantic	Contrato, límites, errores y control de concurrencia
Seguridad	sqlglot	Parseo, sentencia única y solo lectura
Interfaz	Next.js	Formulario, páginas técnicas y proxy al backend
Acceso	Cloudflare	URL HTTPS temporal sin exponer FastAPI directamente
Observabilidad	JSON Lines	Estado, latencia, tamaños y categoría de error
Calidad	pytest, Ruff, TypeScript	Contratos, regresiones y compilación

8.3. Secuencia de una solicitud

Cloudflare recibe la conexión pública y la dirige al frontend, que solo publica rutas esperadas. El proxy de Next.js reenvía la generación a FastAPI. La API valida la estructura antes de adquirir el runtime de GPU. Después de inferir, la guardia SQL analiza el candidato y solo devuelve una consulta de lectura. El evento técnico registra el desenlace sin copiar el contenido proporcionado por el usuario.

Esta separación permite diagnosticar si un fallo pertenece a la interfaz, al contrato, a la carga del modelo o a la salida SQL. También evita publicar el puerto del backend y concentra la interacción externa en

una única puerta HTTPS.

9. Seguridad, privacidad y observabilidad

9.1. Modelo de amenazas

Riesgo	Control	Evidencia
SQL destructivo	Rechazo de DDL, DML y múltiples sentencias	Pruebas y HTTP 422
Entrada fuera de contrato	Límites y presencia de `CREATE TABLE`	Pruebas de API
Fuga en logs	No guardar DDL ni pregunta	Evento anonimizado
Exposición del backend	Escucha solo en `127.0.0.1`	Scripts y puertos verificados
Saturación de GPU	Una generación simultánea	Bloqueo del runtime
Pesos alterados	Revisión y SHA-256 fijados	Registro del adaptador
Código remoto	`trust_remote_code=False` y safetensors	Configuración del runtime

9.2. Guardia de salida

La validación no depende únicamente de buscar palabras prohibidas. `sqlglot` analiza la estructura, normaliza el SQL y comprueba que exista una sola sentencia cuyo árbol corresponda a lectura. Se aceptan `SELECT` y expresiones `WITH ... SELECT`; se rechazan `DELETE`, `UPDATE`, `INSERT`, `DROP`, `ALTER`, `CREATE` y combinaciones múltiples.

La aplicación no ejecuta el SQL contra una base pública. Esta decisión reduce el impacto de una salida incorrecta y permite que el usuario revise el texto antes de copiarlo a un entorno autorizado.

9.3. Telemetría privada

Cada evento incluye fecha UTC, `request_id`, estado, categoría de error, latencia, longitud del esquema, longitud de la pregunta y variante del modelo. No incluye el DDL, la pregunta, credenciales ni resultados de negocio. Los identificadores relacionan una captura y un caso de prueba sin revelar contenido sensible.

Límite de observabilidad

Omitir contenido protege la privacidad, pero reduce el detalle disponible para diagnosticar errores semánticos. En una operación real se necesitaría un proceso voluntario y controlado para que el usuario reporte ejemplos problemáticos sin convertir el log técnico en un almacén de datos.

10. Decisiones de despliegue

10.1. Alternativas evaluadas

Alternativa	Costo	GPU	Estable	Decisión
Hugging Face ZeroGPU	Gratis	Sí	Sí	Bloqueado por antigüedad de cuenta
Hugging Face PRO	USD 9/mes	Sí	Sí	Requiere autorización de pago
Gradio compartido	Gratis	Local	No	Interferido por inspección HTTPS
Cloudflare Quick Tunnel	Gratis	Local	No	Seleccionado y verificado
GPU serverless	Variable	Sí	Sí	Requiere método de pago

El intento de crear un Space ZeroGPU recibió HTTP 402 porque la cuenta tenía menos de 30 días. No se subieron archivos ni se simuló un despliegue permanente. Gradio `share=True` también se descartó tras fallos asociados con la inspección HTTPS local. No se agregó una exclusión a Kaspersky.

Cloudflare permitió reutilizar la interfaz Next.js y la RTX 4050 sin alterar el entorno CUDA. El frontend escucha en `127.0.0.1:3000`; FastAPI permanece en `127.0.0.1:8000` y nunca se publica directamente. La consecuencia aceptada es que la URL cambia y solo existe mientras el equipo mantiene la demostración activa.

10.2. Ruta permanente preparada

`scripts/build_space_bundle.py` genera un paquete de 16 archivos para Gradio y Hugging Face Spaces. El bundle incluye runtime, interfaz, configuración, metadatos del adaptador y dependencias; excluye pesos, datasets, notebooks, logs, capturas, secretos y entornos virtuales. Cuando la cuenta sea elegible para ZeroGPU, este paquete ofrece una ruta estable sin modificar el experimento.

11. Evidencias del pase a producción

11.1. Prueba pública del 5 de noviembre

La demostración publicó el commit `0e304a20f41ed216ae66bc4e535d4a75366cf3a8` mediante Next.js 16.4.3, FastAPI y Cloudflare Quick Tunnel 2026.10.7. La página respondió HTTP 200 y el backend reportó modo `model`, variante QLoRA y adaptador local listo.

Caso	Resultado observado	Latencia
Fácil	Lista nombre y ciudad de clientes ordenados	43,039 ms
Medio	Clientes sin pedidos mediante anti-join	3,014 ms
Difícil	Cliente con mayor total completado en 2024	8,030 ms
`DELETE`	Rechazado por la API con HTTP 422	–
Sentencia múltiple	Rechazada por la API con HTTP 422	–

La primera latencia incluye la carga fría del modelo; los siguientes casos representan un runtime caliente. La diferencia muestra por qué la exposición debe iniciar el servicio con anticipación y no interpretar los 43 segundos como el tiempo normal de todas las solicitudes.

11.2. Capturas y cierre controlado

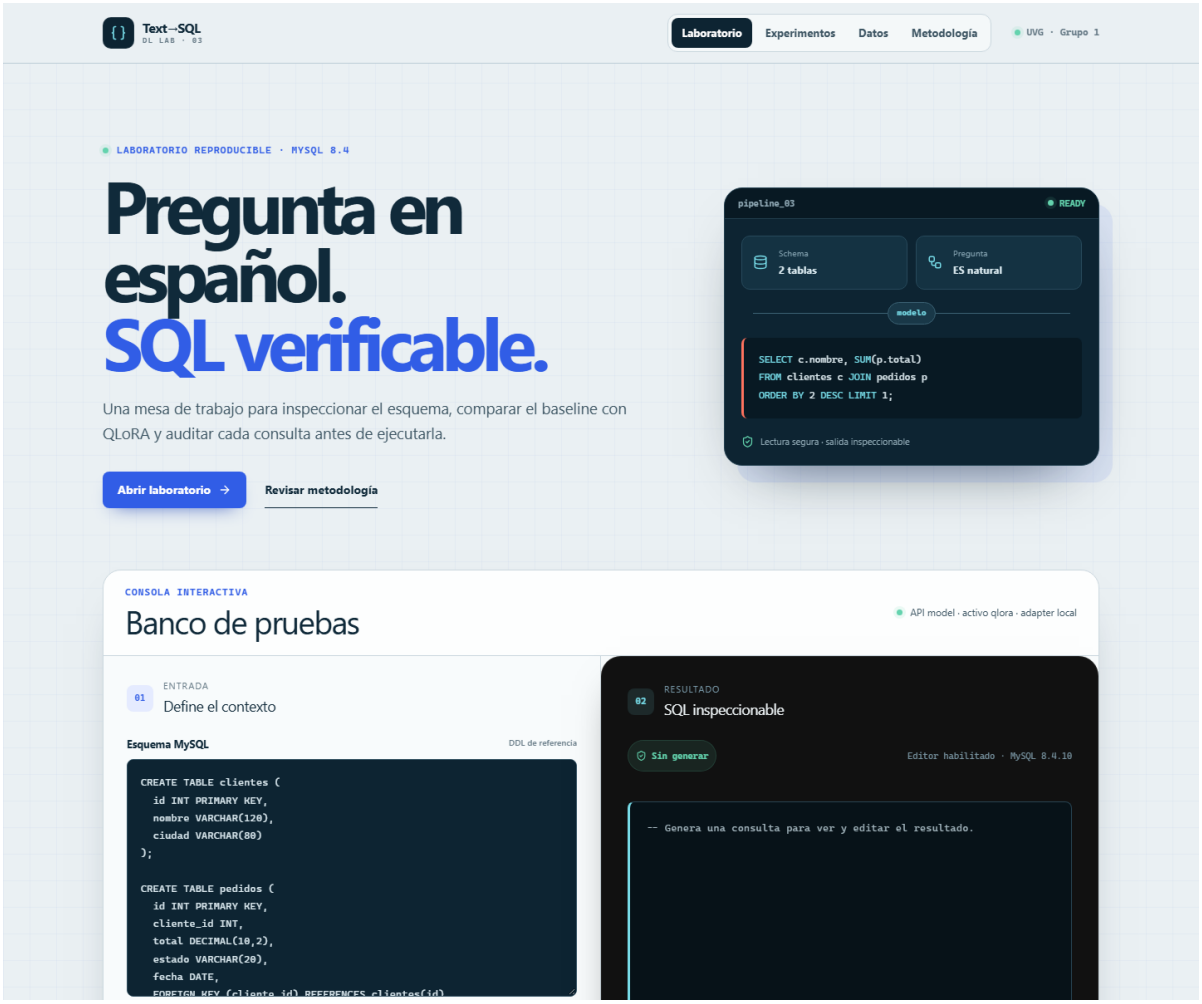


Figura 2: Interfaz conectada al backend en modo modelo, con QLoRA activo y adaptador local.

La URL `https://bytes-aside-abraham-troy.trycloudflare.com` fue válida únicamente durante la ventana documentada de 16:13:10 a 16:17:26, hora de Guatemala. Al finalizar se detuvieron el túnel, Next.js y FastAPI; luego se confirmó que los puertos 3000, 7860, 8000 y 20241 estaban libres. Por ello, un error `DNS_PROBE_FINISHED_NXDOMAIN` posterior es el comportamiento esperado de una URL temporal cerrada.

11.3. Inventario de evidencia

Evidencia	Ubicación	Estado
Suite automatizada	`tests/`	43 aprobadas
Smoke local RTX 4050	`docs/evidencias/pruebas/`	65.1 s, 1,472 MB
Evento anonimizado	`docs/evidencias/logs/`	Registrado
Aplicación pública	`docs/evidencias/despliegue.md`	HTTP 200
Capturas	`docs/evidencias/capturas/`	Dos revisadas
Resultados sintéticos	`docs/evidencias/pruebas/`	3 de 3 correctos
Controles destructivos	`docs/evidencias/pruebas/`	2 de 2 rechazados

12. Verificación y reproducibilidad

12.1. Batería de calidad

Control	Cobertura	Resultado
pytest	API, seguridad, configuración, datos, bundle y auditorías	43/43
Ruff	Runtime, backend, pruebas y scripts finales	Sin errores
Next.js build	TypeScript, rutas estáticas y proxy dinámico	Aprobado
Bundle de Space	Lista permitida de archivos	16 archivos
Auditor final	Requisitos, PDFs, evidencia y scripts	Aprobado
Revisión visual	Informe, presentación y capturas	Aprobada

La suite incluye contratos de entrada y salida, rechazo de SQL inseguro, configuración fijada, integridad del dataset, selección del adaptador, importación de scripts, construcción del bundle y presencia de artefactos finales. Las pruebas no sustituyen la inferencia real; por eso se complementan con el smoke test de GPU y la prueba pública.

12.2. Preparación desde una clonación limpia

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-demo.txt
pnpm.cmd --dir web install --frozen-lockfile
```

Para iniciar, verificar y cerrar la demostración:

```
.\scripts\demo\start_demo.ps1
.\scripts\demo\verify_demo.ps1 -RunInference
.\scripts\demo\stop_demo.ps1
```

El primer comando construye Next.js, inicia FastAPI y genera una URL nueva. El segundo comprueba HTTP, inferencias y rechazos. El tercero detiene únicamente los procesos cuyos identificadores, nombres y horas de inicio coinciden con el estado de `.demo-run/`. Para una prueba sin URL pública puede usarse `-LocalOnly`.

12.3. Comandos de auditoría

```
python -m pytest -q
python -m ruff check production.py app.py backend text2sql tests
pnpm.cmd --dir web build
python scripts/build_space_bundle.py
python scripts/audit_delivery.py --final
python scripts/audit_final.py --final
```

El modelo base, la revisión, el adaptador, la semilla y la habilitación de QLoRA se fijan en `config.json`. Los pesos recuperables no se incluyen en Git, pero su origen, revisión y hash sí. Esta combinación permite reconstruir el servicio sin depender de una copia opaca en la computadora del equipo.

13. Limitaciones, operación y trabajo futuro

13.1. Limitaciones conocidas

- La validación propia contiene 40 preguntas sobre únicamente dos esquemas inéditos.
- Los datos son sintéticos y no representan todos los patrones de una organización real.
- El sistema no ejecuta la consulta ni valida permisos sobre una base de negocio.
- Una consulta segura puede ser semánticamente incorrecta o ineficiente.
- La GPU local limita la concurrencia y exige que la computadora anfitriona permanezca activa.
- Quick Tunnel produce una URL temporal y no constituye disponibilidad permanente.
- La omisión de contenido en logs limita el diagnóstico de errores semánticos.

13.2. Condiciones para mantenerlo en producción

La recomendación al comité es mantener el modelo para consultas supervisadas, no como ejecutor autónomo. La continuidad se condiciona a conservar la restricción de solo lectura, exigir revisión antes de ejecutar, monitorear latencia y errores, y evaluar por separado cualquier dominio nuevo.

En una operación más amplia se deberían definir un catálogo de dominios aceptados, límites de servicio, política de retención de telemetría, evaluación periódica, revisión de dependencias y un proceso de rollback para el adaptador. También conviene desplegar el bundle en una GPU estable cuando la cuenta sea elegible, repetir las 40 consultas en ese entorno y comparar latencia y calidad con la evidencia local.

13.3. Próximos pasos priorizados

1. obtener una URL permanente con GPU compatible;
2. ejecutar una evaluación por cada dominio real autorizado;
3. agregar métricas agregadas de error y latencia sin registrar contenido;
4. definir objetivos de disponibilidad y tiempos de respuesta;

5. documentar revisión, aprobación y ejecución segura del SQL generado;
6. repetir pruebas de seguridad y dependencias antes de cada release.

14. Conclusiones

El proyecto cumplió el objetivo de llevar a producción un modelo existente sin presentar como nueva la experimentación del Proyecto 3. La adaptación QLoRA conserva su 97.4 % de EX ponderada y se integra en una ruta completa: interfaz, API, GPU, validación de salida, telemetría y evidencia.

La prueba pública demostró accesibilidad externa, inferencia real y rechazo de casos destructivos. La reproducibilidad se sostiene con revisiones y hashes fijados, scripts de demostración, pruebas automatizadas, documentación y un paquete mínimo de despliegue. La URL histórica no permanece activa por diseño; cada demostración genera una nueva y la cierra al terminar.

La decisión recomendada es mantener el modelo bajo supervisión humana. La calidad observada justifica seguir utilizándolo, pero la cobertura limitada, el cambio de dominio y la posibilidad de errores semánticos impiden tratarlo como un sistema autónomo. El valor del proyecto está precisamente en combinar la capacidad del modelo con controles explícitos y evidencia auditable.

Uso de inteligencia artificial

OpenAI Codex apoyó el análisis del enunciado, la migración, la implementación, las pruebas y la documentación. v0 se utilizó como punto de partida visual de la interfaz del Proyecto 3. Los integrantes revisan los cambios y son responsables de explicar el código, las métricas y las decisiones. Ninguna API externa procesa las solicitudes del servicio; la inferencia utiliza el modelo y el adaptador abiertos descritos en este documento.

Mapa de artefactos

Ruta	Propósito
`README.md`	Documento técnico principal y guía de ejecución
`config.json`	Revisiones, adaptador y semilla
`production.py`	Puerta de producción y telemetría
`backend/`	API, contrato y servicio de inferencia
`web/`	Interfaz Next.js y proxy
`text2sql/`	Prompt, runtime, evaluación y seguridad SQL
`resultados/`	Métricas, corridas, manifiestos y hashes
`docs/evidencias/`	Capturas, logs y verificaciones fechadas
`scripts/demo/`	Inicio, verificación y cierre reproducibles
`tests/`	Suite automatizada de 43 pruebas